VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

Ojas Manojkumar Saundatti (1WA24CS196)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by OJAS MANOJKUMAR SSAUNDATTI (1WA24CS196), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of an OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

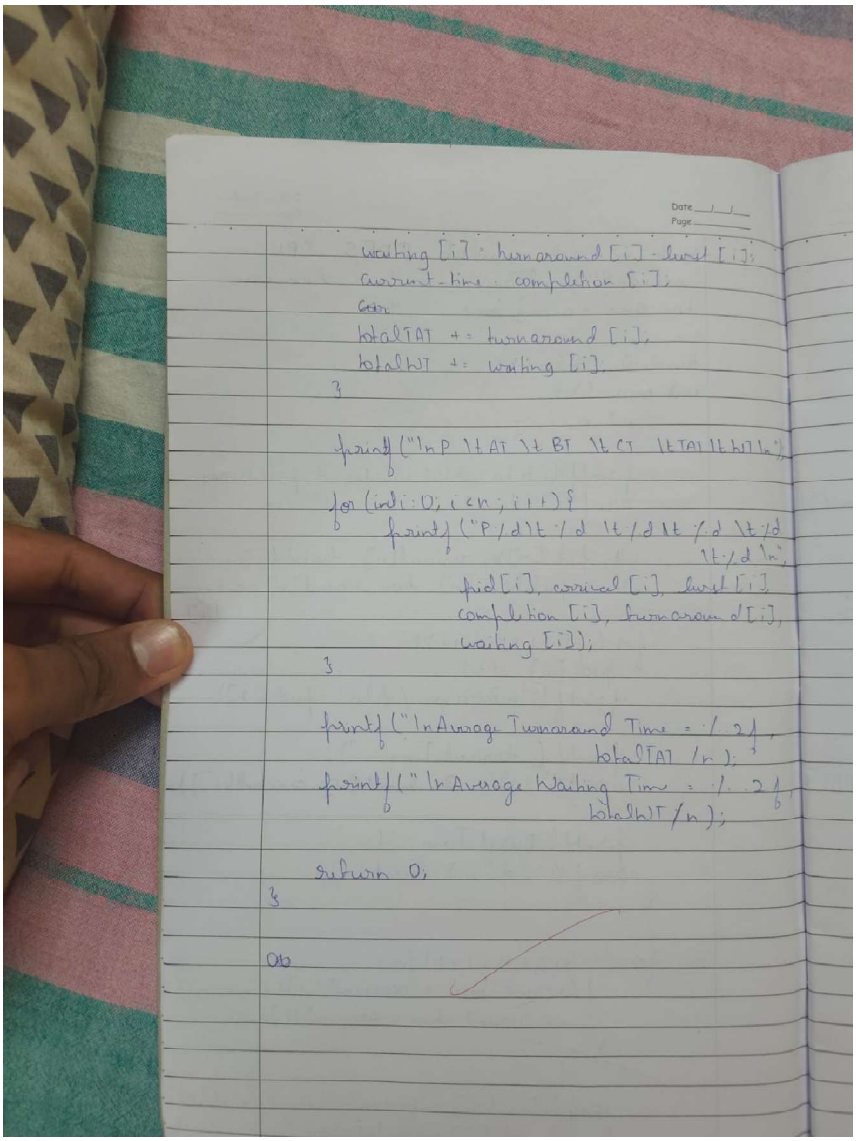
Sl No	Experiment Title	Page No
1	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-15
2	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	15-25
3	Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one) a) Rate-Monotonic b) Earliest-Deadline First c) Proportional Scheduling	26-41

4	Write a C program to simulate: (Any one) a) Producer-Consumer problem using semaphores b) Dining-Philosopher's problem	41-64
5	Write a C program to simulate: (Any one) a) Bankers' algorithm for the purpose of deadlock avoidance b) Deadlock Detection	65-79
6	Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-Fit b) Best-Fit c) First-Fit	79-89
7	Write a C program to simulate page replacement algorithms. (Any one) a) FIFO b) LRU c) Optimal	90-98

FCFS

Write a C to simulate FCFS CPU scheduling algorithm to find turnaround time and waiting time

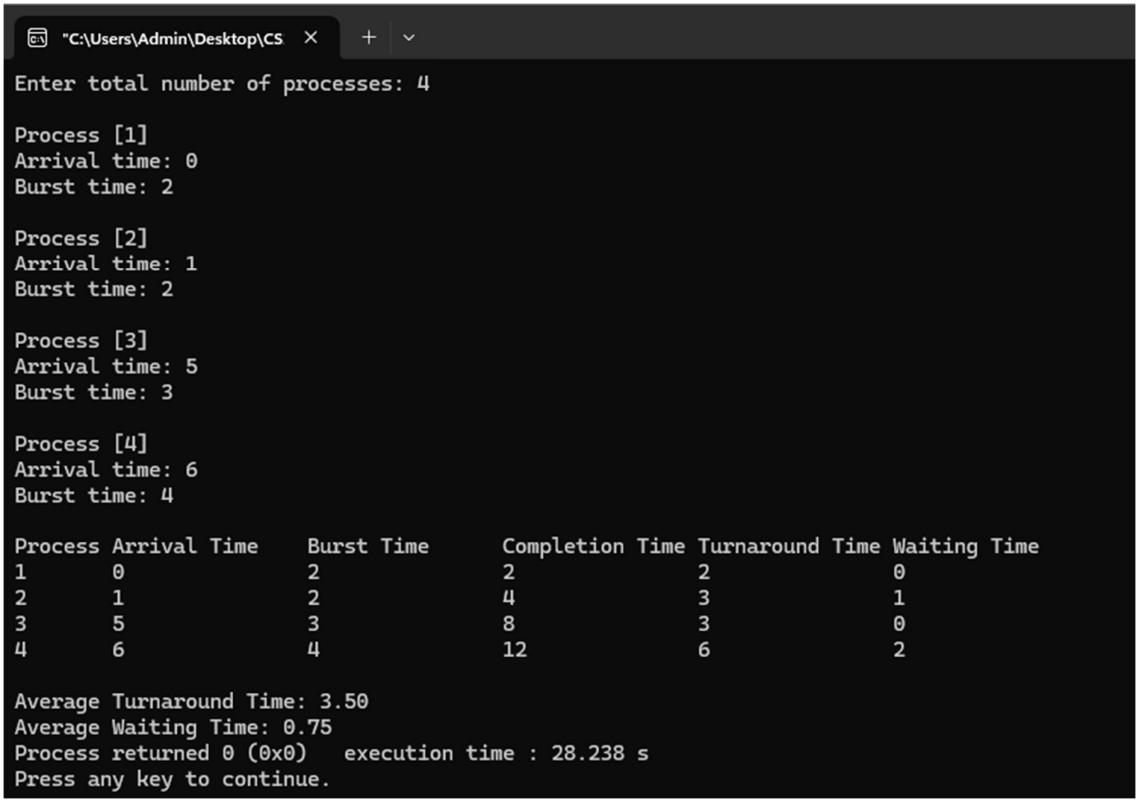
```
#include <stdio.h>
int main () {
    int n, i, j;

    printf ("Enter total number of processes ");
    scanf ("%d", &n);

    int pid [n], arrival[n], burst [n];
    int completion [n], turnaround [n], waiting [n];

    for (i=0; i<n, i++) {
      pid [i] = i+1;
      printf ("\n Process %d \n", pid [i]);

      printf ("Arrival Time: ");
      scanf ("%d", &burst [i], arrival[i]);

      printf ("Burst Time: ");
      scanf ("%d", &burst [i]);
    }

    for (i=0, i<n; i++) {
        if (current-time < arrival [i])
            current-time = arrival [i]
    }

    completion [i] = current_time + burst [i];
    turnaround [i] = completion [i] - arrival [i];
```




```
#include <stdio.h>

int main() {
   int n, i;

   printf("Enter total number of processes: ");
   scanf("%d",&n);

   int at[n], bt[n], ct[n], tat[n], wt[n];
```

```
float avg_tat = 0, avg_wt = 0;

for(i=0;i<n;i++) {
   printf("\nProcess [%d]\n", i+1);

   printf("Arrival time: ");
   scanf("%d",&at[i]);

   printf("Burst time: ");
   scanf("%d",&bt[i]);
}

ct[0] = at[0] + bt[0];

for(i=1;i<n;i++) {
   if(ct[i-1] < at[i])
      ct[i] = at[i] + bt[i];
   else
      ct[i] = ct[i-1] + bt[i];
}

for(i=0;i<n;i++) {
   tat[i] = ct[i] - at[i];
   wt[i] = tat[i] - bt[i];

   avg_tat += tat[i];
   avg_wt += wt[i];
}
```

```
    avg_tat = avg_tat / n;
    avg_wt = avg_wt / n;

    printf("\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround Time\tWaiting Time\n");

    for(i=0;i<n;i++) {
        printf("%d\t%d\t\t%d\t\t%d\t\t%d\t\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage Turnaround Time: %.2f", avg_tat);
    printf("\nAverage Waiting Time: %.2f", avg_wt);

    return 0;
}
```

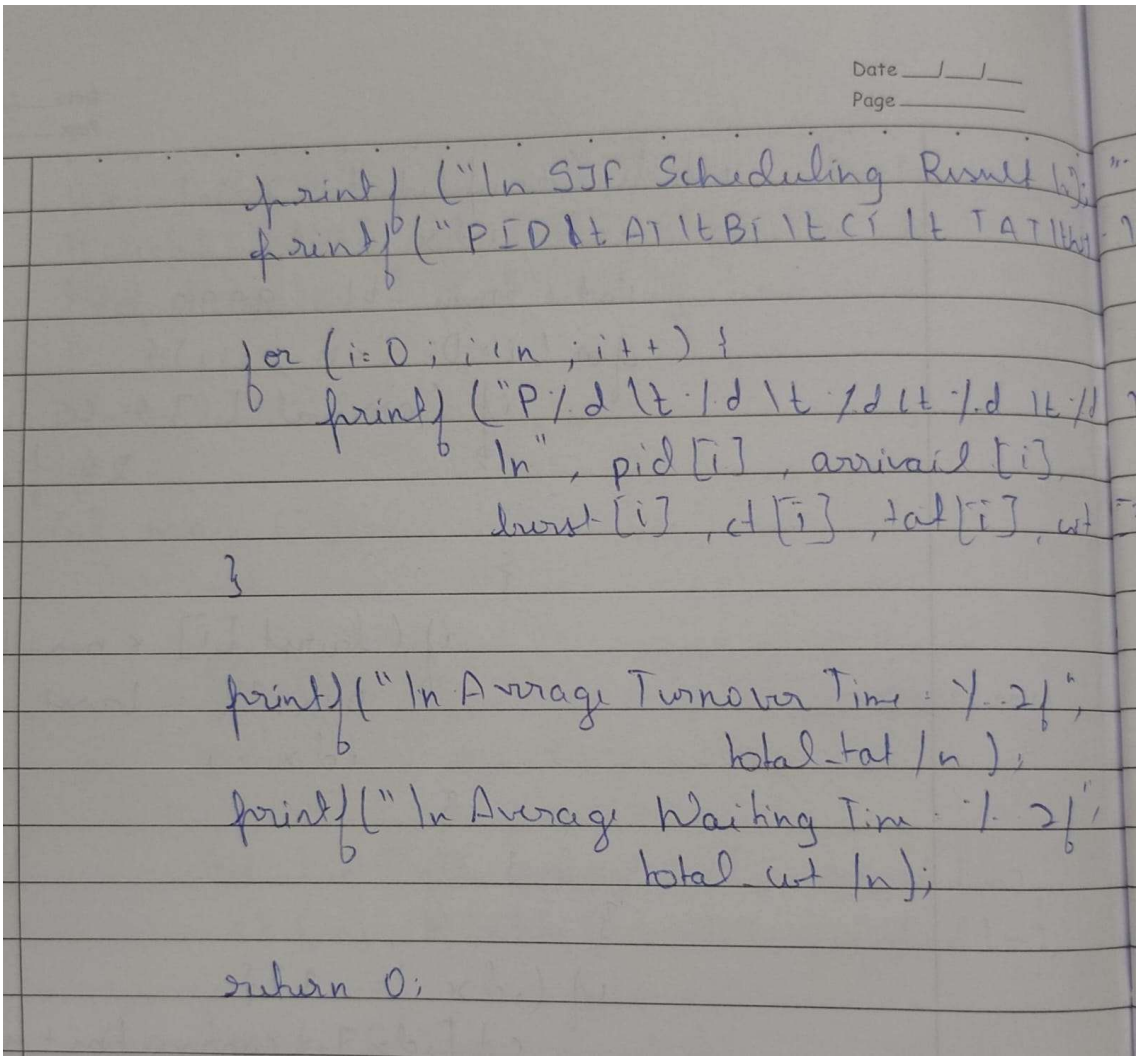


SJF_NonPreemptive

Write a C Program to simulate Non Preemptive SJF scheduling algorithm to find turnaround and waiting time.

```
#include <stdio.h>
int
int main () {
    int n, i;

    printf ("Enter the number of processes:");
    scanf (" %d", &n);

    int pid[n], arrival[n], burst[n],
    ct[n], tat[n], wt[n], finished[n];

    for (i=0; i<n; i++) {
        pid[i] = i+1;
        finished[i] = 0;

        printf ("\n Process %d \n", pid[i]

        printf ("Arrival Time : ");
        scanf (" %d", &arrival[i]);

        printf (" Burst Time : ");
        scanf (" %d", &Burst[i]);
    }
int
id completed = 0, current_time = 0,
float total_tat = 0, total_wt = 0;
```

```
while (completed < n) {
    int idx = -1;
    int min_bt = 9999;
    for (i=0; i<n; i++) {
        if (arrival[i] <= current_time
                && finished[i]
                == 0)
        {
            if (burst[i] < min_bt) {
                min_bt = burst[i];
                idx = i;
            }
        }
    }
    if (idx != -1) {
        ct[idx] = current_time + burst[idx];
        tat[idx] = ct[idx] - arrival[idx];
        wt[idx] = tat[idx] - burst[idx];

        current_time = ct[idx];
        finished[idx] = 1;
        completed++;

        total_tat += tat[idx];
        total_wt += wt[idx];
    } else {
        current_time++;
    }
}
```

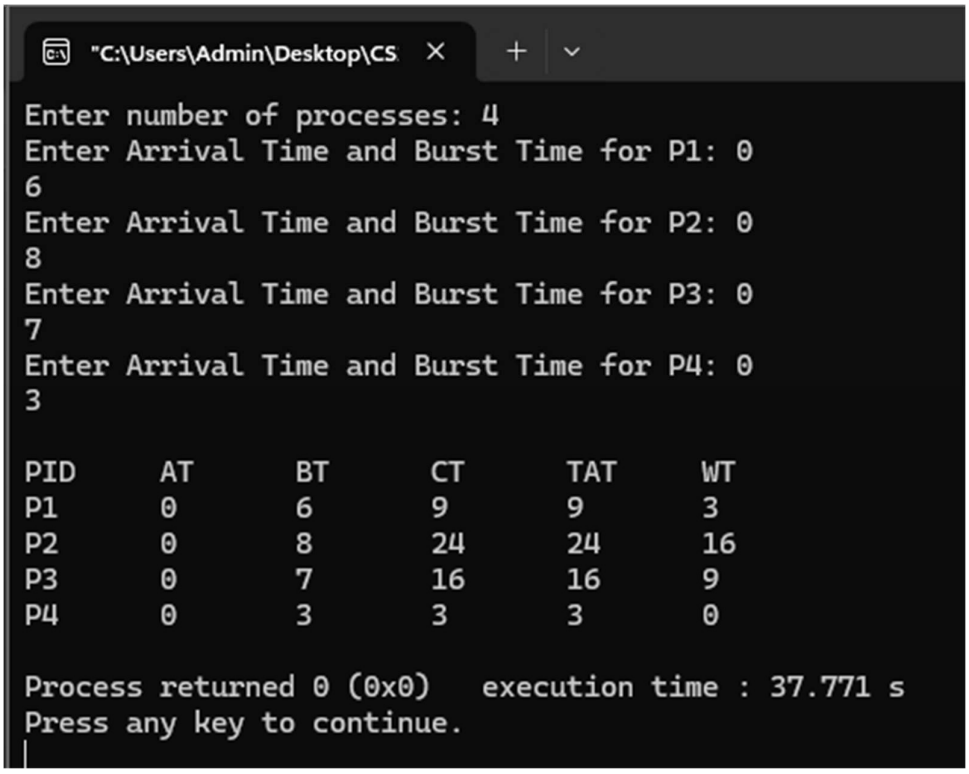


```
printf("\n SJF Scheduling Result \n");
printf("PID\t AT\t BT\t CT\t TAT\t WT\n");

for(i=0; i<n; i++) {
    printf("P%d\t %d\t %d\t %d\t %d\t %d\n",
        pid[i], arrival[i],
        burst[i], ct[i], tat[i], wt[i]);
}

printf("\n Average Turnaround Time : %.2f",
                    total_tat/n);
printf("\n Average Waiting Time : %.2f",
                    total_wt/n);

return 0;
```

```
#include <stdio.h>

int main() {
   int n, i, j, current_time = 0, completed = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   int pid[n], at[n], bt[n], ct[n], tat[n], wt[n], finished[n];

   for(i = 0; i < n; i++)
```

```
    finished[i] = 0;

for(i = 0; i < n; i++) {
    printf("Enter Arrival Time and Burst Time for P%d: ", i+1);
    pid[i] = i + 1;
    scanf("%d %d", &at[i], &bt[i]);
}

while(completed < n) {
    int idx = -1;
    int min_bt = 100000;

    for(i = 0; i < n; i++) {
        if(at[i] <= current_time && finished[i] == 0) {
            if(bt[i] < min_bt) {
                min_bt = bt[i];
                idx = i;
            }
        }
    }

    if(idx == -1) {
        current_time++;
    } else {
        ct[idx] = current_time + bt[idx];
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];
```

```
            current_time = ct[idx];
            finished[idx] = 1;
            completed++;
        }
    }

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}
```



SJF_Preemptive

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time SJF Preemptive

```
#include <stdio.h>
#include <limits.h>

int main()
{
    int n, i;
    int pid[20], arrival[20], burst[20];
    int remaining[20], completion[20];
    turnaround[20], waiting[20];
    int finished[20];

    int current_time = 0, completed = 0;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        pid[i] = i+1;
        finished[i] = 0;

        printf("\n Process %d \n", pid[i]);
        printf("\n");
        printf(" Arrival Time : ");
        scanf("%d", &arrival[i]);
```

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time SJF Preemptive

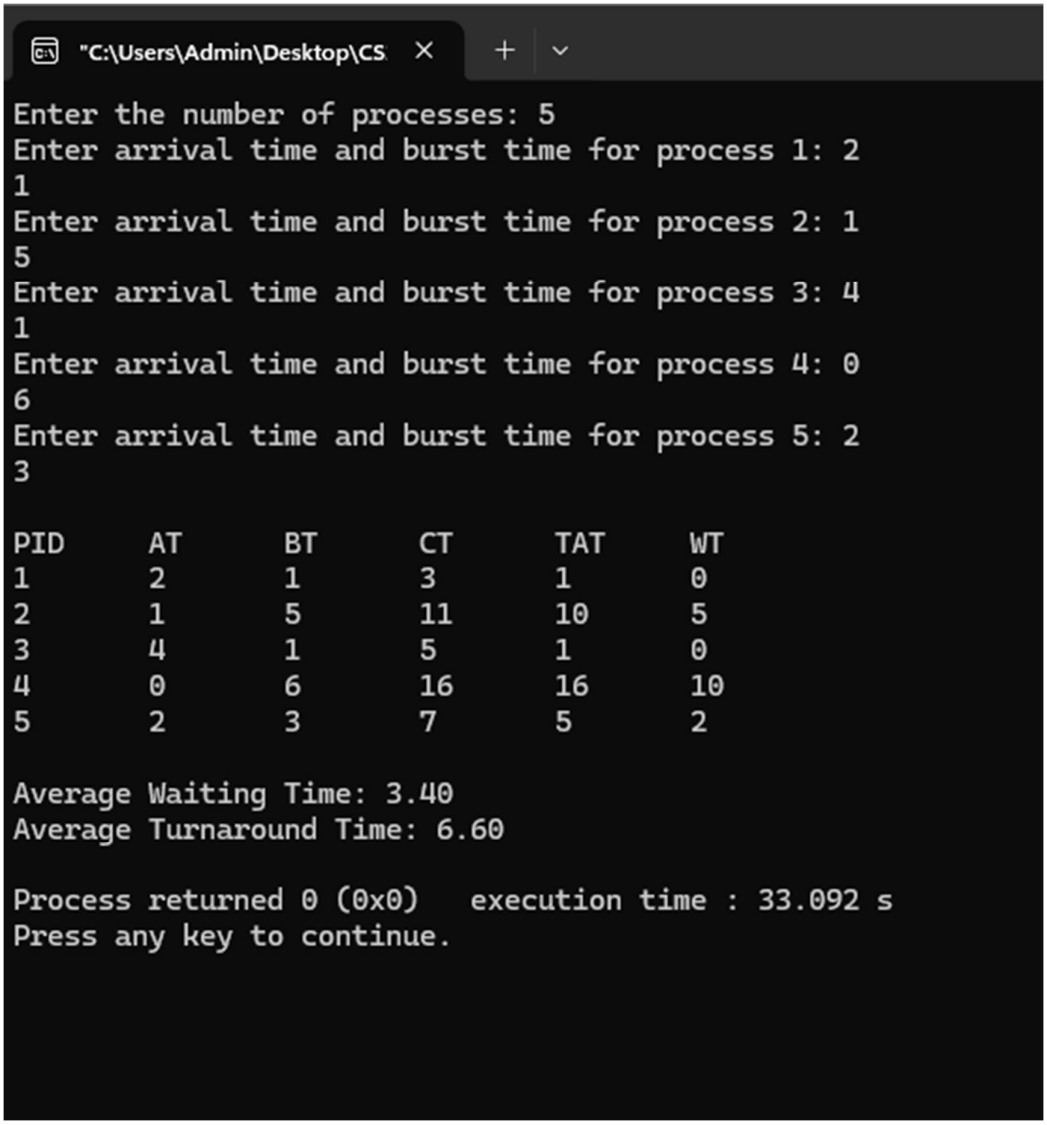
```
#include <stdio.h>
#include <limits.h>

int main()
{
    int n, i;
    int pid[20], arrival[20], burst[20];
    int remaining[20], completion[20];
    turnaround[20], waiting[20];
    int finished[20];

    int current_time = 0, completed = 0;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        pid[i] = i+1;
        finished[i] = 0;

        printf("\n Process %d \n", pid[i]);
        printf("\n");
        printf(" Arrival Time : ");
        scanf("%d", &arrival[i]);
```

```
        printf ("Burst Time : ");
        scanf("%d", &burst[i]);

        remaining[i] = burst[i];
    }

    while(completed < n)
    {
        int shortest = -1;
        int min = INT_MAX;

        for (i=0; i<n; i++)
        {
            if (arrival[i] <= current_time
            && finished[i] == 0 &&
            remaining[i] < min)
            {
            min = remaining[i];
            shortest = i;
            }
        }

        if (shortest == -1)
        {
            current_time++;
            continue;
        }

        if remaining[shortest] --;
        current_time++;
```

```c
#include <stdio.h>

int main() {
   int n, i, time = 0, completed = 0, shortest = -1;

   printf("Enter the number of processes: ");
   scanf("%d", &n);

   int pid[n], at[n], bt[n], rt[n], ct[n], tat[n], wt[n];
   int min_rt;

   for(i = 0; i < n; i++) {
      pid[i] = i + 1;
      printf("Enter arrival time and burst time for process %d: ", pid[i]);
      scanf("%d %d", &at[i], &bt[i]);
      rt[i] = bt[i];   // Remaining time initially equals burst time
   }

   while(completed < n) {
      shortest = -1;
      min_rt = 9999;

      for(i = 0; i < n; i++) {
         if(at[i] <= time && rt[i] > 0 && rt[i] < min_rt) {
            min_rt = rt[i];
            shortest = i;
         }
      }
```

```
    if(shortest == -1) {
        time++;
        continue;
    }

    rt[shortest]--;
    time++;

    if(rt[shortest] == 0) {
        completed++;
        ct[shortest] = time;
        tat[shortest] = ct[shortest] - at[shortest];
        wt[shortest] = tat[shortest] - bt[shortest];
    }
}

float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
          pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);

    avg_wt += wt[i];
    avg_tat += tat[i];
}
```

```
    avg_wt /= n;
    avg_tat /= n;

    printf("\nAverage Waiting Time: %.2f", avg_wt);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat);

    return 0;
}
```

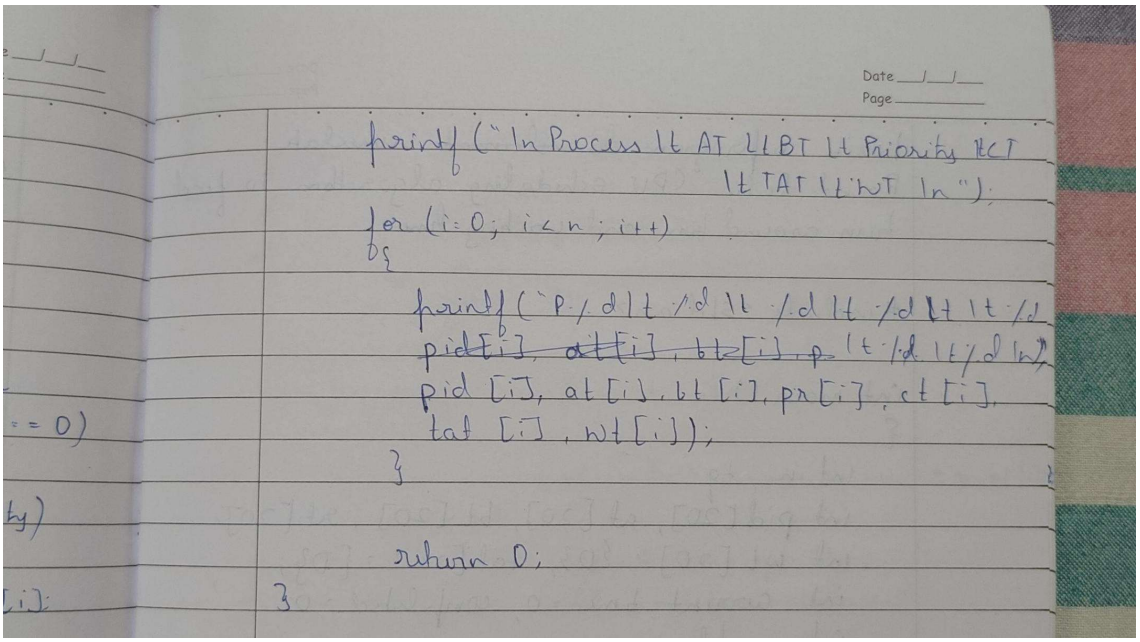


Priority scheduling non-pre-emptive

Write a program to simulate Priority non-preemptive CPU scheduling algorithm to find turn around time and waiting time.

```
#include <stdio.h>

int main()
{
    int n;
    int pid[20], at[20], pr[20];
    int ct[20], tat[20], wt[20];
    int completed[20] = {0};

    int current_time = 0, completed_count = 0;
    int i;

    printf("Enter number of process: ");
    scanf("%d", &n);

    for(i=0; i<n; i++)
    {
        printf("\n Process %d \n", i+1);
        printf("Enter Arrival Time: ");
        scanf("%d", &at[i]);

        printf("Enter Burst Time: ");
        scanf("%d", &bt[i]);

        printf("Enter Priority: ");
        scanf("%d", &pr[i]);
    }
```

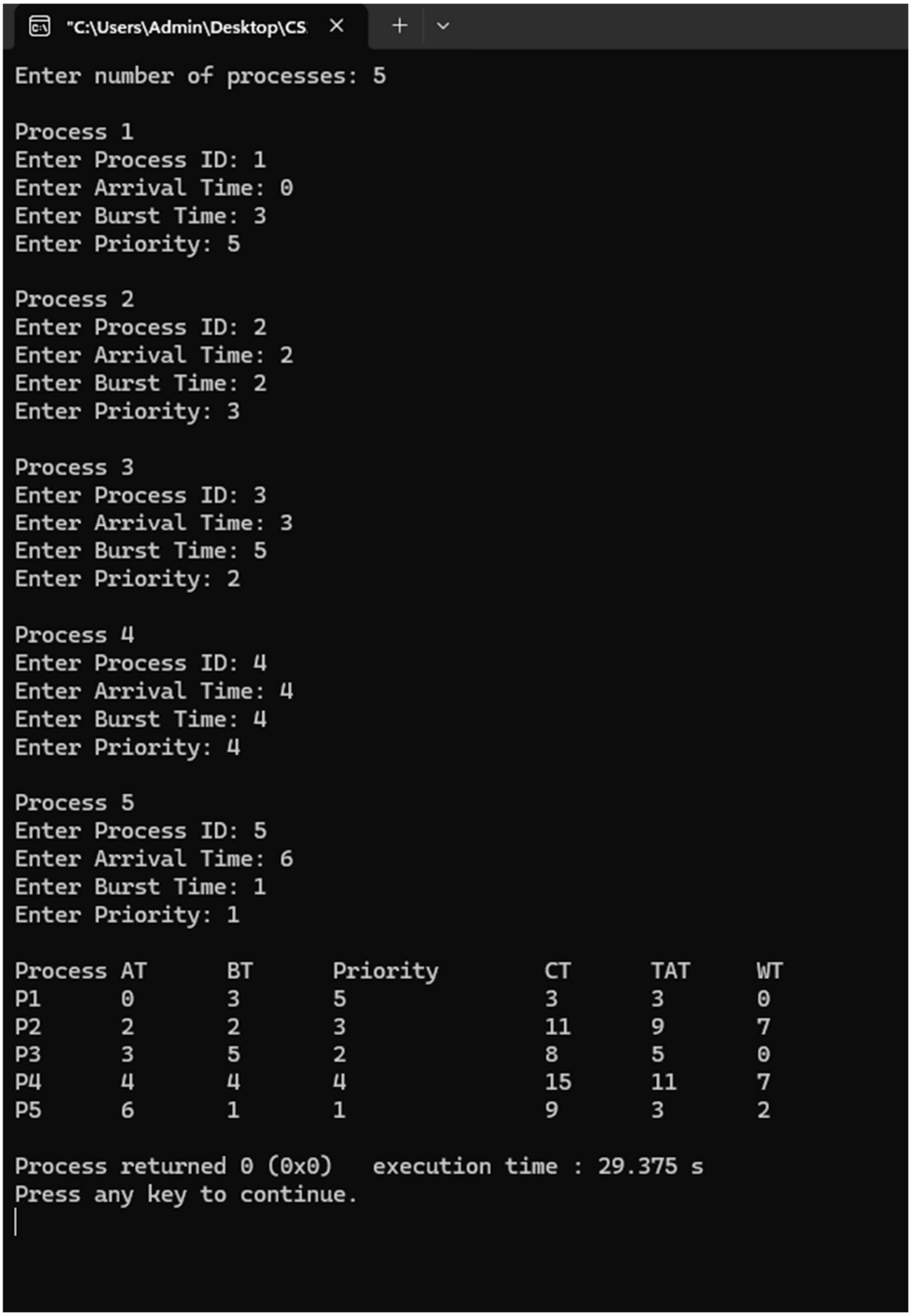
```
while (completed_count < n)
{
    int highest_priority = 9999;
    int selected_process = -1;
    for (i=0; i<n; i++)
    {
        if (at[i] <= current_time &&
                completed[i] == 0)
        {
            if (pr[i] < highest_priority)
            {
                highest_priority = pr[i];
                selected_process = i;
            }
        }
    }

    if (selected_process == -1)
    {
        current_time++;
    }
    else
    {
        ct[selected_process] = current_time
            + bt[selected_process];
        tat[selected_process] = ct[selected
        _process] - at[selected_process];
        wt[selected_process] = tat[
        selected_process] - bt[selected_process];

        current_time = ct[selected_process];
        completed[selected_process] = 1;
        completed_count++;
    }
}
```




```
#include <stdio.h>

int main()
{
   int n;
   int pid[20], at[20], bt[20], pr[20];
   int ct[20], tat[20], wt[20];
   int completed[20] = {0};

   int current_time = 0, completed_count = 0;
   int i;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
   {
      printf("\nProcess %d\n", i+1);
      printf("Enter Process ID: ");
      scanf("%d", &pid[i]);
```

```
    printf("Enter Arrival Time: ");
    scanf("%d", &at[i]);

    printf("Enter Burst Time: ");
    scanf("%d", &bt[i]);

    printf("Enter Priority: ");
    scanf("%d", &pr[i]);
}

while(completed_count < n)
{
    int highest_priority = 9999;
    int selected_process = -1;

    for(i = 0; i < n; i++)
    {
        if(at[i] <= current_time && completed[i] == 0)
        {
            if(pr[i] < highest_priority)
            {
                highest_priority = pr[i];
                selected_process = i;
            }
        }
    }

    if(selected_process == -1)
    {
        current_time++;
```

```
        }
        else
        {
            ct[selected_process] = current_time + bt[selected_process];
            tat[selected_process] = ct[selected_process] - at[selected_process];
            wt[selected_process] = tat[selected_process] - bt[selected_process];

            current_time = ct[selected_process];
            completed[selected_process] = 1;
            completed_count++;
        }
    }

    printf("\nProcess\tAT\tBT\tPriority\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\n",
        pid[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}
```


"C:\Users\Admin\Desktop\CS
Enter number of processes: 5
Process 1
Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Priority: 5
Process 2
Enter Process ID: 2
Enter Arrival Time: 2
Enter Burst Time: 2
Enter Priority: 3
Process 3
Enter Process ID: 3
Enter Arrival Time: 3
Enter Burst Time: 5
Enter Priority: 2
Process 4
Enter Process ID: 4
Enter Arrival Time: 4
Enter Burst Time: 4
Enter Priority: 4
Process 5
Enter Process ID: 5
Enter Arrival Time: 6
Enter Burst Time: 1
Enter Priority: 1
Process AT BT Priority CT TAT WT
P1 0 3 5 3 3 0
P2 2 2 3 11 9 7
P3 3 5 2 8 5 0
P4 4 4 4 15 11 7
P5 6 1 1 9 3 2
Process returned 0 (0x0) execution time : 29.375 s
Press any key to continue.

Priority scheduling Pre-emptive

Write a program to simulate Priority Pre-emptive CPU scheduling algorithm to find turn around time and waiting time

```
#include <stdio.h>
int main()
{
    int n, i;
    int pid[20], at[20], bt[20], pr[20];
    int rt[20], ct[20], tat[20], wt[20];

    int current_time = 0, completed = 0;
    int highest_priority, selected_process;

    printf("Enter number of Process is: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\n Process %d \n", i+1);

        printf("Enter process ID: ");
        scanf("%d", &pid[i]);

        printf("Enter Arrival Time: ");
        scanf("%d", &at[i]);

        printf("Enter Burst Time: ");
        scanf("%d", &bt[i]);

        printf("Enter Priority: ");
        scanf("%d", &pr[i]);
```

```
        rt[i] = bt[i];
    }

    while (completed < n)
    {
        highest_priority = 9999;
        selected_process = -1;

        for (i=0; i<n; i++)
        {
            if (at[i] <= current_time && rt[i] > 0)
            {
                if (px[i] < highest_priority)
                {
                    highest_priority = px[i];
                    selected_process = i;
                }
            }
        }
        if (selected_process == -1)
        {
            current_time++;
        }
        else
        {
            rt[selected_process]--;
            current_time++;

            if (rt[selected_process] == 0)
            {
                ct[selected_process] = current_time+1
```

```
        tat[selected] = ct[selected] - at[selected];
        wt[selected] = tat[selected] - bt[selected];
        avg_tat += tat[selected];
        avg_wt += wt[selected];

        completed++;
    }
  }
}

printf("\n Priority Scheduling (Pre-Emptive) :\n");
printf("PID\t Prior\t AT\t BT\t CT\t TAT\t WT\t RT\n");

for(i=0; i<n; i++) {

    printf("P%d\t %d\t %d\t %d\t %d\t %d\t %d\t %d\n",
    pid[i], p[i], at[i], bt[i], ct[i], tat[i], wt[i], wt[i]);
}

printf("\n Average turnaround time: %.6f\n", avg_tat/n);
printf("Average waiting time: %.6f\n", avg_wt/n);

return 0;

}
```

```
#include <stdio.h>

int main()
{
   int n, choice;
   int pid[20], pr[20], at[20], bt[20], rt[20];
   int ct[20], tat[20], wt[20];
   int completed = 0, current_time = 0;
   int i, selected, best_priority;

   float avg_wt = 0, avg_tat = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter priorities (1 for higher number means higher priority, 2 for lower number means higher
priority): ");
   scanf("%d", &choice);

   printf("Enter priorities:\n");
   for(i=0;i<n;i++)
      scanf("%d",&pr[i]);

   printf("Enter arrival times:\n");
   for(i=0;i<n;i++)
      scanf("%d",&at[i]);

   printf("Enter burst times:\n");
   for(i=0;i<n;i++)
   {
      scanf("%d",&bt[i]);
      rt[i] = bt[i];
```

```
    pid[i] = i+1;
}

while(completed < n)
{
    selected = -1;

    if(choice == 2)
        best_priority = 9999;
    else
        best_priority = -1;

    for(i=0;i<n;i++)
    {
        if(at[i] <= current_time && rt[i] > 0)
        {
            if(choice == 2)
            {
                if(pr[i] < best_priority)
                {
                    best_priority = pr[i];
                    selected = i;
                }
            }
            else
            {
                if(pr[i] > best_priority)
                {
                    best_priority = pr[i];
                    selected = i;
                }
```

```
        }
      }
    }

    if(selected == -1)
    {
      current_time++;
    }
    else
    {
      rt[selected]--;
      current_time++;

      if(rt[selected] == 0)
      {
        ct[selected] = current_time;
        tat[selected] = ct[selected] - at[selected];
        wt[selected] = tat[selected] - bt[selected];

        avg_tat += tat[selected];
        avg_wt += wt[selected];

        completed++;
      }
    }
  }

  printf("\nPriority Scheduling (Pre-Emptive):\n");
  printf("PID\tPrior\tAT\tBT\tCT\tTAT\tWT\tRT\n");

  for(i=0;i<n;i++)
```

```
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i], pr[i], at[i], bt[i], ct[i], tat[i], wt[i], wt[i]);
    }

    printf("\nAverage turnaround time: %.6f\n", avg_tat/n);
    printf("Average waiting time: %.6f\n", avg_wt/n);

    return 0;
}
```

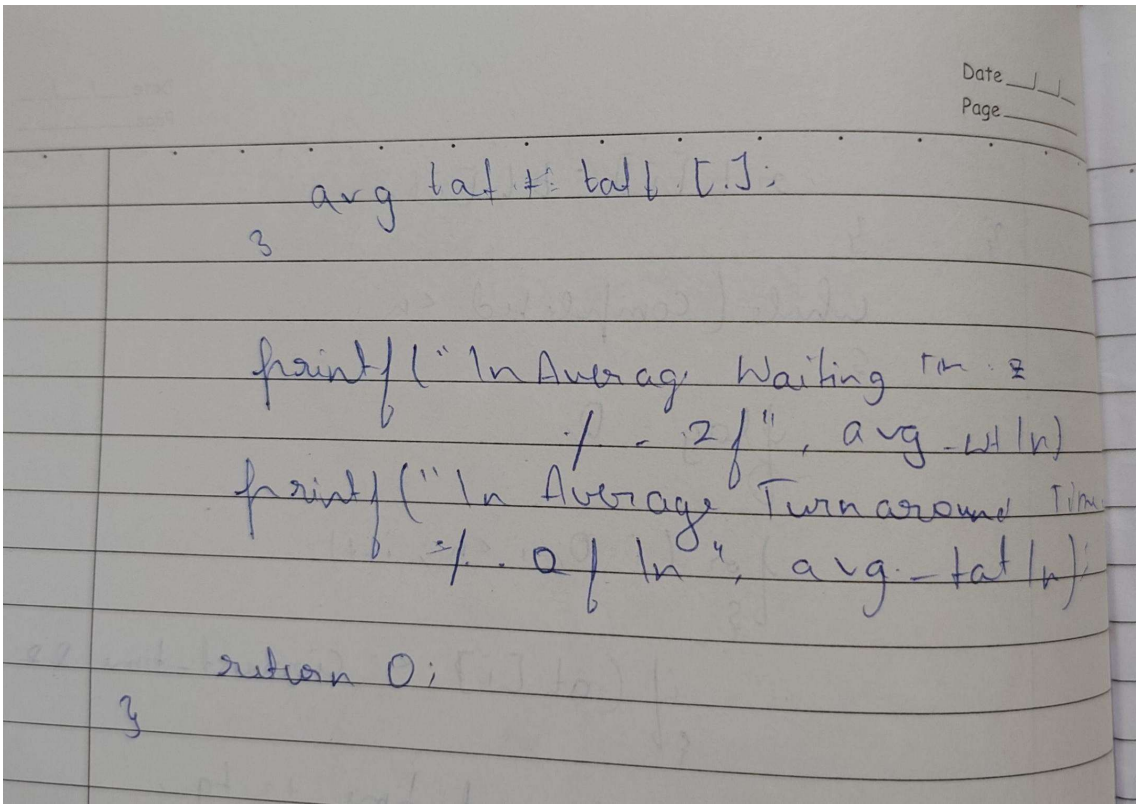
```
Enter number of processes: 7
Enter priorities (1 for higher number means higher priority, 2 for lower number means higher priority): 2
Enter priorities:

3
4
4
5
2
6
1
Enter arrival times:
0
1
3
4
5
6
10
Enter burst times:
8
2
4
1
6
5
1

Priority Scheduling (Pre-Emptive):
PID     Prior   AT      BT      CT      TAT     WT      RT
P1      3       0       8       15      15      7       7
P2      4       1       2       17      16      14      14
P3      4       3       4       21      18      14      14
P4      5       4       1       22      18      17      17
P5      2       5       6       12      7       1       1
P6      6       6       5       27      21      16      16
P7      1       10      1       11      1       0       0

Average turnaround time: 13.714286
Average waiting time: 9.857142

Process returned 0 (0x0)   execution time : 75.354 s
Press any key to continue.
```

Round Robin

Write a program to ~~simulate~~ simulate Round Robin CPU scheduling algorithm to find turn around time and waiting time

```
#include <stdio.h>

int main()
{
    int n, tq;
    int pid[20], at[20], bt[20], rt[20];
    int wt[20] = {0}, tat[20] = {0};
    int current_time = 0, completed = 0;
    int i, flag;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    for(i = 0; i < n; i++)
    {
        printf("\n Process %d \n", i+1);

        printf("Enter process ID: ");
        scanf("%d", &pid[i]);

        printf("Enter Arrival Time: ");
        scanf("%d", &at[i]);

        printf("Enter Burst Time: ");
        scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i];
    }
    while(completed < n)
    {
        flag = 0;
        for(i=0; i<n; i++)
        {
            if(at[i] <= current_time && rt[i]
            {
                current_time += tq;
                rt[i] -= tq;
            }
            else
            {
                current_time += rt[i];
                tat[i] = current_time - at[i];
                wt[i] = tat[i] - bt[i];
                rt[i] = 0;
                completed++;
            }
        }
        if(flag == 0)
        {
            current_time++;
        }
    }
    printf("\nPID\tAT\tBT\tWT\tTAT\n");
    for(i=0; i<n; i++)
    {
        avg_wt += wt[i];
```




```
#include <stdio.h>

int main()
{
   int n, tq;
   int at[20], bt[20], rt[20];
   int ct[20], tat[20], wt[20];
   int i, time = 0, remain, flag = 0;

   float avg_wt = 0, avg_tat = 0;

   printf("Enter number of processes: ");
   scanf("%d",&n);

   printf("Enter Time Quantum: ");
   scanf("%d",&tq);
```

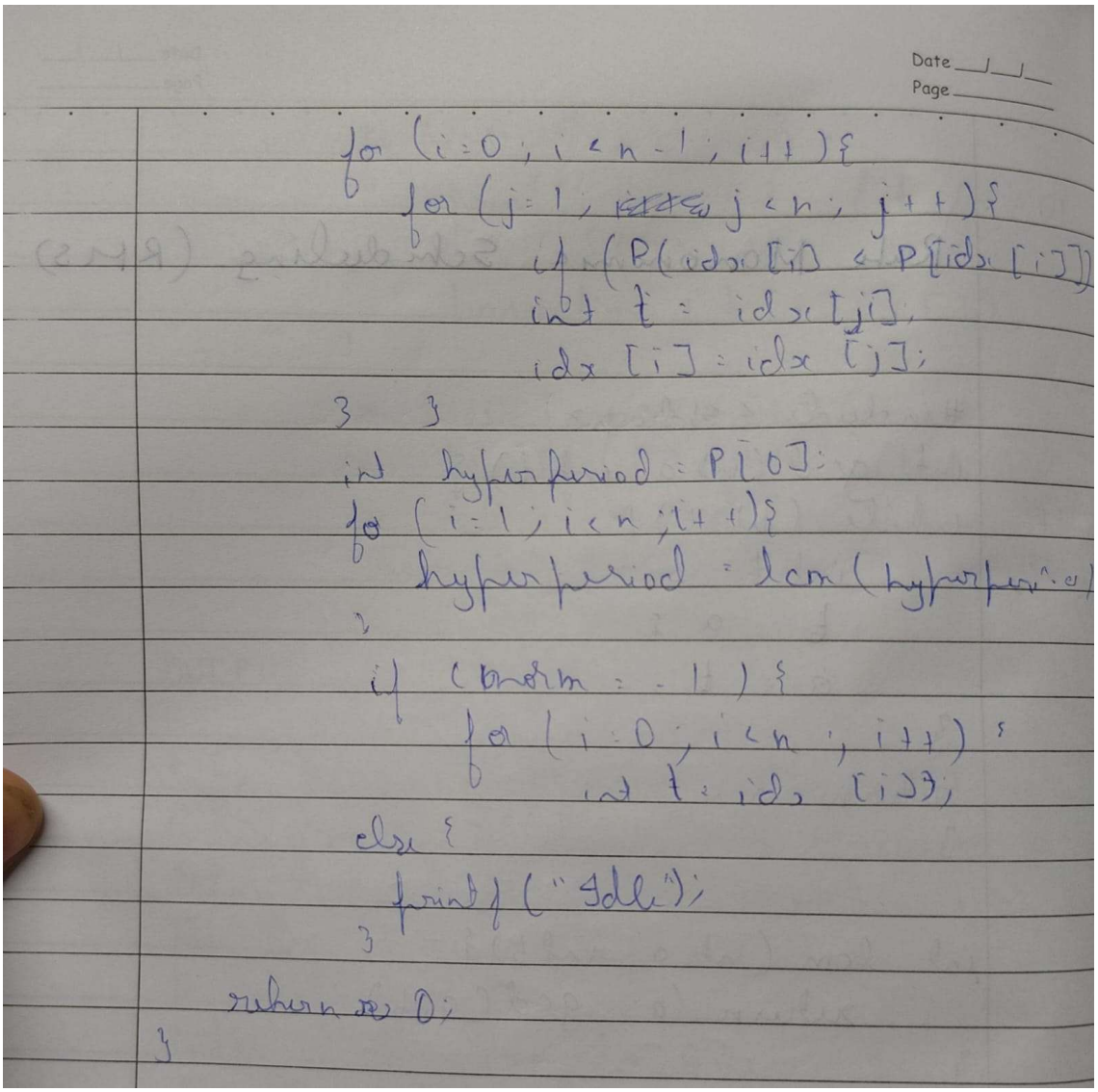
```
printf("Enter arrival times:\n");
for(i=0;i<n;i++)
   scanf("%d",&at[i]);

printf("Enter burst times:\n");
for(i=0;i<n;i++)
{
   scanf("%d",&bt[i]);
   rt[i] = bt[i];
}

remain = n;

for(time=0,i=0; remain!=0;)
{
   if(rt[i] <= tq && rt[i] > 0)
   {
      time += rt[i];
      rt[i] = 0;
      flag = 1;
   }
   else if(rt[i] > 0)
   {
      rt[i] -= tq;
      time += tq;
   }

   if(rt[i]==0 && flag==1)
   {
      remain--;
      ct[i] = time;
```

```
            tat[i] = time - at[i];
            wt[i] = tat[i] - bt[i];

            avg_wt += wt[i];
            avg_tat += tat[i];

            flag = 0;
        }

        if(i == n-1)
            i = 0;
        else if(at[i+1] <= time)
            i++;
        else
            i = 0;
    }

    printf("\nRRS scheduling:\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

    for(i=0;i<n;i++)
    {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }

    printf("\nAverage turnaround time: %.6fms\n", avg_tat/n);
    printf("\nAverage waiting time: %.6fms\n", avg_wt/n);

    return 0;
}
```

```
Enter number of processes: 5
Enter Time Quantum: 2
Enter arrival times:
0
1
2
3
4
Enter burst times:
5
3
1
2
3

RRS scheduling:
PID     AT      BT      CT      TAT     WT      RT
1       0       5       14      14      9       0
2       1       3       12      11      8       0
3       2       1       5       3       2       0
4       3       2       7       4       2       0
5       4       3       13      9       6       0

Average turnaround time: 8.200000ms

Average waiting time: 5.400000ms

Process returned 0 (0x0)   execution time : 14.973 s
Press any key to continue.
```

Earliest Deadline first

```
#include <stdio.h>
int gcd (int a, int b) {
while (b != 0) {
    int t = b
    b = a ;
    a = t ;
}
return a;
}

int lcm (int a, int b) {
    return (a, gcd(a,b)
}

int main() {
    int n;
    printf("Enter the number of task : ");
    scanf("%d", &n);
    int c[20], P[20], idx[20];
    int i, j;

    for(i=0; i<n ; i++) {
        printf("Enter computation time %d:
                                    i+1);
        scanf(" Enter period P%d", i+1);
        scanf("%d", &P[i]);
        idx[i] = i;
    }
```




```
#include <stdio.h>


int main() {

   printf("Name:Ojas Manojkumar Ssundatti\n" "USN:1WA24CS196\n" "PRG:Earliest Deadline first (EDF) Algorithm\n");

   int n, i;


   printf("Enter number of tasks: ");

   scanf("%d", &n);
```

```
int bt[n], dl[n], p[n];
int rem[n], abs_dl[n];

printf("Enter Burst times:\n");
for (i = 0; i < n; i++) {
   scanf("%d", &bt[i]);
   rem[i] = 0;
}

printf("Enter Deadlines:\n");
for (i = 0; i < n; i++) {
   scanf("%d", &dl[i]);
}

printf("Enter Periods:\n");
for (i = 0; i < n; i++) {
   scanf("%d", &p[i]);
}



for (i = 0; i < n; i++) {
   abs_dl[i] = dl[i];
}

// Total time
int total_time = p[0];
for (i = 1; i < n; i++) {
   if (p[i] > total_time)
```

```
    total_time = p[i];
}

printf("\nScheduling occurs for %d ms\n\n", total_time);

int prev_idle = 0;

for (int time = 0; time < total_time; time++) {


  for (i = 0; i < n; i++) {
    if (time % p[i] == 0) {
      rem[i] = bt[i];
      abs_dl[i] = time + dl[i];
    }
  }

  int selected = -1;
  int min_deadline = 9999;


  for (i = 0; i < n; i++) {
    if (rem[i] > 0) {
      if (abs_dl[i] < min_deadline) {
        min_deadline = abs_dl[i];
        selected = i;
      }
    }
```

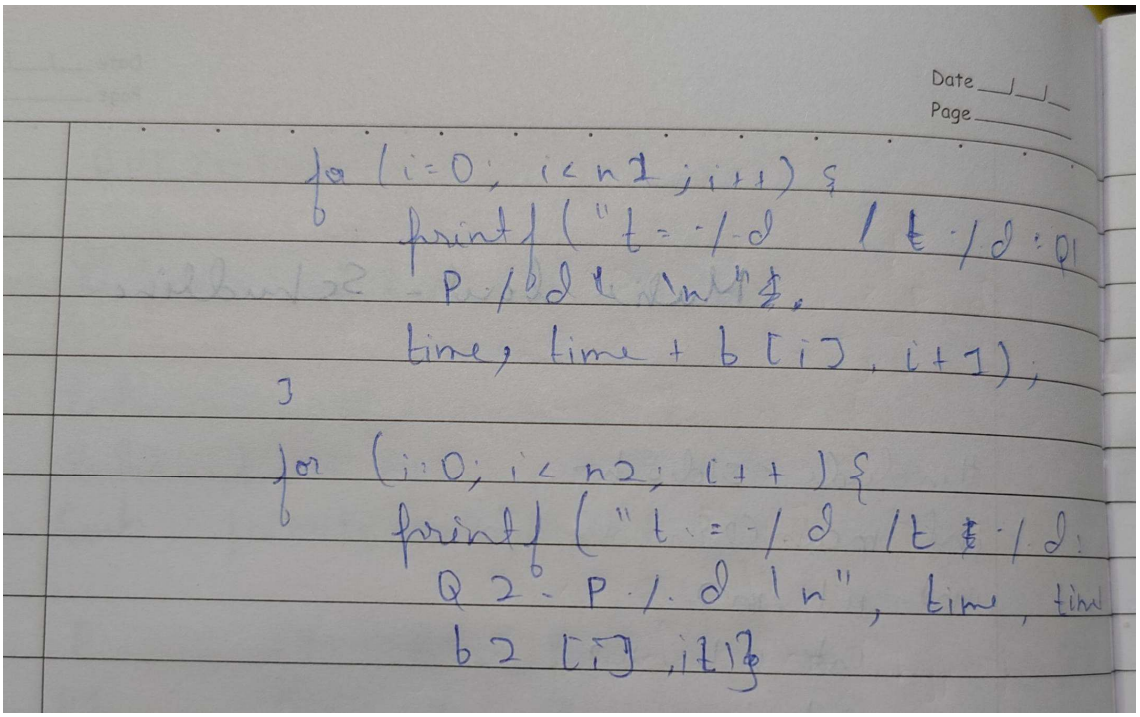
```
    }

    if (selected == -1) {
      if (!prev_idle) {
        printf("%dms onwards : CPU is idle.\n", time);
        prev_idle = 1;
      }
    } else {
      printf("%dms : Task %d is running.\n", time, selected + 1);
      rem[selected]--;
      prev_idle = 0;
    }
  }

  return 0;
}
```

OUTPUT

```
Name:Ojas Manojkumar Ssundatti
USN:1WA24CS196
PRG:Earliest Deadline first (EDF) Algorithm

Enter number of tasks: 3
Enter Burst times:
1 2 1
Enter Deadlines:
4 5 7
Enter Periods:
4 5 7

Scheduling occurs for 7 ms

0ms : Task 1 is running.
1ms : Task 2 is running.
2ms : Task 2 is running.
3ms : Task 3 is running.
4ms : Task 1 is running.
5ms : Task 2 is running.
6ms : Task 2 is running.
```

Proportional Share Scheduling

```
#include <stdio.h>
int main(){
int n1, n2;
    int n1, n2;
    int i
    printf("Enter number of process in
                    queue 1: ");
    scanf("%d", &n1);
    printf("Enter number of process in
Sca                 queue 2: ");
    scanf("%d", &n2);

int b1[20], b2[20];
printf("Enter burst time for queue 1:");
for(i=0; i<n1; i++){
        printf("P1 - P%d:", i+1);
        scanf("%d", &b1[i]);
}
printf("Enter burst time for queue 2: ");
for(i=0; i<n2; i++){
        printf("P2 - P%d", i+1);
        scanf("%d", &b2[i]);
}
int time=0;
printf("In Execution order: \n");
```




```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
   char name[5];
   int tickets;
} Process;

int main() {
   printf("Name:Ojas Manojkumar Ssundatti\n" "USN:1WA24CS196\n"
"PRG:Proportional share scheduling\n");
   int n, total_tickets = 0;
   float total_T = 0.0;

   printf("Enter the number of Processes: ");
```

```
scanf("%d", &n);

Process p[n];

srand(time(NULL));

for (int i = 0; i < n; i++) {
   printf("\nProcess %d:\n", i + 1);

   sprintf(p[i].name, "P%d", i + 1);

   printf("Tickets: ");
   scanf("%d", &p[i].tickets);

   total_tickets += p[i].tickets;
   total_T += p[i].tickets;
}

printf("\n--- Proportional Share Scheduling ---\n");

printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d", &m);

for (int i = 0; i < m; i++) {
   int winning_ticket = rand() % total_tickets + 1;
   int accumulated_tickets = 0;
   int winner_index;
```

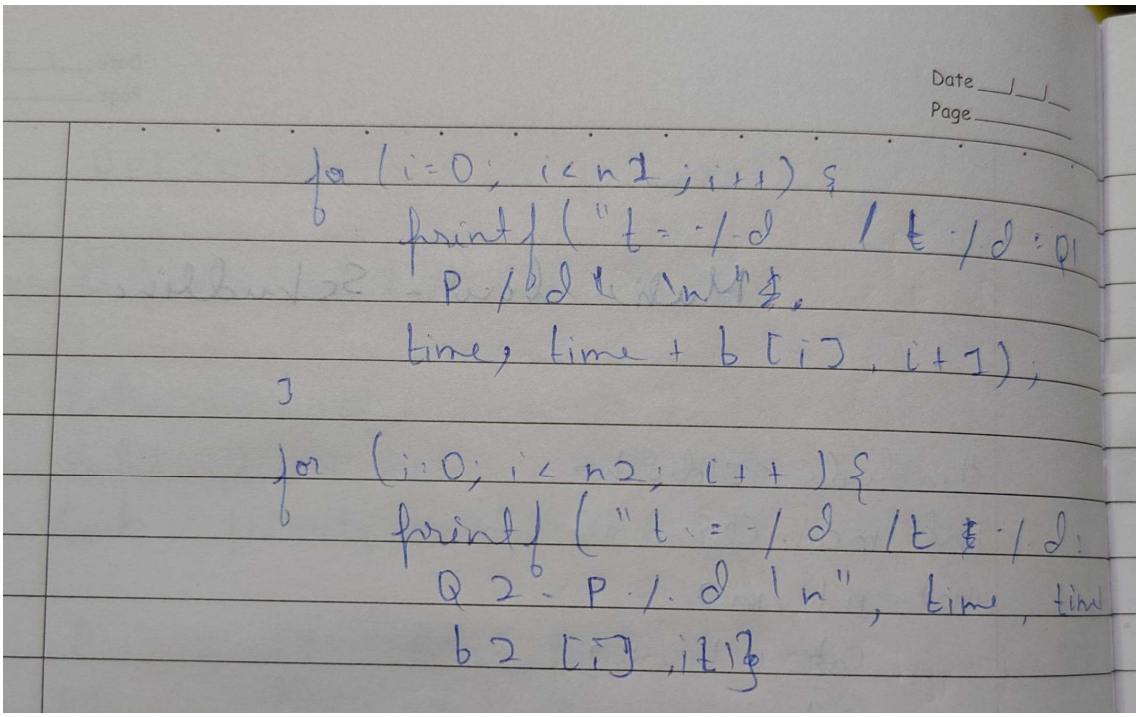
```
    for (int j = 0; j < n; j++) {
        accumulated_tickets += p[j].tickets;

        if (winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
        }
    }

    printf("Tickets picked: %d, Winner: %s\n",
        winning_ticket, p[winner_index].name);
  }

  for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n",
        p[i].name,
        ((p[i].tickets / total_T) * 100));
  }

  return 0;
}
```

OUTPUT

```
Name:Ojas Manojkumar Ssundatti
USN:1WA24CS196
PRG:Proportional share scheduling

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 10

Tickets picked: 12, Winner: P2
Tickets picked: 55, Winner: P3
Tickets picked: 8, Winner: P1
Tickets picked: 41, Winner: P3
Tickets picked: 27, Winner: P2
Tickets picked: 58, Winner: P3
Tickets picked: 31, Winner: P3
Tickets picked: 16, Winner: P2
Tickets picked: 49, Winner: P3
Tickets picked: 5, Winner: P1

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.
```

Rate monotonic scheduling

```
#include <stdio.h>
int main(){
int n1, n2;
    int n1, n2;
    int i
    printf("Enter number of process in
                    queue 1: ");
    scanf("%d", &n1);
    printf("Enter number of process in
Sca                 queue 2: ");
    scanf("%d", &n2);

int b1[20], b2[20];
printf("Enter burst time for queue 1:");
for(i=0; i<n1; i++){
        printf("P1 - P%d:", i+1);
        scanf("%d", &b1[i]);
}
printf("Enter burst time for queue 2: ");
for(i=0; i<n2; i++){
        printf("P2 - P%d", i+1);
        scanf("%d", &b2[i]);
}
int time=0;
printf("In Execution order: \n");
```




```
#include <stdio.h>
#include <math.h>

int gcd(int a, int b) {
  while (b != 0) {
    int t = b;
    b = a % b;
    a = t;
  }
  return a;
}

int lcm(int a, int b) {
  return (a * b) / gcd(a, b);
}
```

```
int main() {
   printf("Name:Ojas Manojkumar Ssundatti\n" "USN:1WA24CS196\n" "PRG:Rate monotonic scheduling (RMS) Algorithm\n");
   int n, i;

   printf("Enter the number of processes:");
   scanf("%d", &n);

   int bt[n], p[n];

   printf("Enter the CPU burst times:\n");
   for (i = 0; i < n; i++) {
      scanf("%d", &bt[i]);
   }

   printf("Enter the time periods:\n");
   for (i = 0; i < n; i++) {
      scanf("%d", &p[i]);
   }

   int hyper = p[0];
   for (i = 1; i < n; i++) {
      hyper = lcm(hyper, p[i]);
   }

   printf("LCM=%d\n\n", hyper);

   printf("Rate Monotone Scheduling:\n");
```

```
printf("PID\tBurst\tPeriod\n");
for (i = 0; i < n; i++) {
   printf("%d\t%d\t%d\n", i + 1, bt[i], p[i]);
}

float U = 0;
for (i = 0; i < n; i++) {
   U += (float)bt[i] / p[i];
}

float bound = n * (pow(2, (float)1/n) - 1);

printf("\n%f <= %f => %s\n",
     U, bound, (U <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n", hyper);

int rem[n];
for (i = 0; i < n; i++)
   rem[i] = 0;

printf("\n");

int prev = -2;

for (int time = 0; time < hyper; time++) {

   for (i = 0; i < n; i++) {
```

```
        if (time % p[i] == 0) {
            rem[i] = bt[i];
        }
    }

    int selected = -1;

    for (i = 0; i < n; i++) {
        if (rem[i] > 0) {
            if (selected == -1 || p[i] < p[selected]) {
                selected = i;
            }
        }
    }

    if (selected != prev) {
        if (selected == -1)
            printf("%dms onwards: CPU is idle\n", time);
        else
            printf("%dms onwards: Process %d running\n", time, selected + 1);
    }

    if (selected != -1) {
        rem[selected]--;
    }

    prev = selected;
}
```

```
    return 0;
}
```

OUTPUT

```
Name:Ojas Manojkumar Ssundatti
USN:1WA24CS196
PRG:Rate monotonic scheduling (RMS) Algorithm

Enter the number of processes:3

Enter the CPU burst times:
1 2 2

Enter the time periods:
8 5 10

LCM=40

Rate Monotone Scheduling:
PID     Burst   Period
1       1       8
2       2       5
3       2       10

0.725000 <= 0.779763 => true
Scheduling occurs for 40 ms

0ms onwards: Process 2 running
2ms onwards: Process 1 running
3ms onwards: Process 3 running
5ms onwards: Process 2 running
7ms onwards: CPU is idle
8ms onwards: Process 1 running
9ms onwards: CPU is idle
10ms onwards: Process 2 running
12ms onwards: Process 3 running
14ms onwards: CPU is idle
15ms onwards: Process 2 running
17ms onwards: CPU is idle
18ms onwards: Process 1 running
19ms onwards: CPU is idle
20ms onwards: Process 2 running
22ms onwards: Process 3 running
24ms onwards: Process 1 running
25ms onwards: Process 2 running
27ms onwards: CPU is idle
30ms onwards: Process 2 running
32ms onwards: Process 3 running
34ms onwards: CPU is idle
35ms onwards: Process 2 running
37ms onwards: CPU is idle
```

Producer Consumer using queue

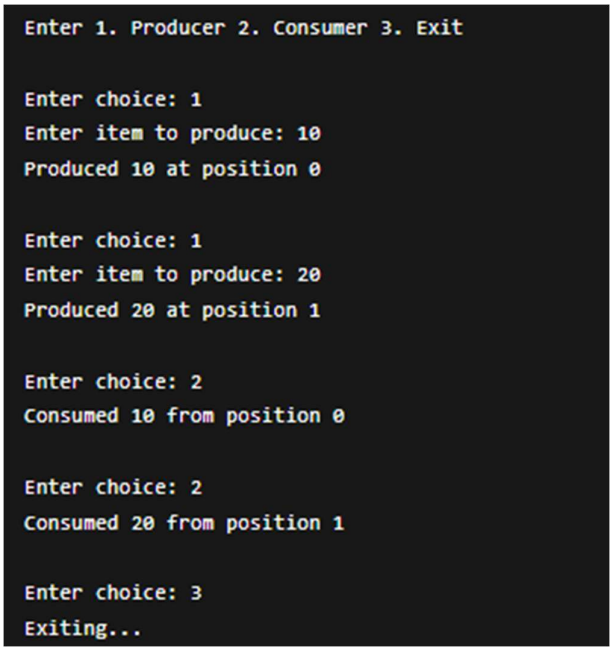
```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int front = -1, rear = -1;

sem_t empty, full, mutex;

void producer() {
   int item;

   if ((rear + 1) % BUFFER_SIZE == front) {
      printf("Buffer is FULL! Cannot produce.\n");
      return;
   }

   printf("Enter item to produce: ");
   scanf("%d", &item);

   sem_wait(&empty);
   sem_wait(&mutex);

   if (front == -1) front = 0;
```

```
    rear = (rear + 1) % BUFFER_SIZE;
    buffer[rear] = item;

    printf("Produced %d at position %d\n", item, rear);

    sem_post(&mutex);
    sem_post(&full);
}

void consumer() {
    int item;

    if (front == -1) {
        printf("Buffer is EMPTY! Cannot consume.\n");
        return;
    }

    sem_wait(&full);
    sem_wait(&mutex);

    item = buffer[front];
    printf("Consumed %d from position %d\n", item, front);

    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % BUFFER_SIZE;
    }
```

```
    sem_post(&mutex);
    sem_post(&empty);
}

int main() {
    int choice;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    printf("Enter ");
    printf("1. Producer ");
    printf("2. Consumer ");
    printf("3. Exit\n");

    while (1) {
        printf("\nEnter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                producer();
                break;
            case 2:
                consumer();
                break;
```

```
        case 3:
            printf("Exiting...\n");
            sem_destroy(&empty);
            sem_destroy(&full);
            sem_destroy(&mutex);
            exit(0);
        default:
            printf("Invalid choice!\n");
    }
  }

  return 0;
}
```

OUTPUT



Producer Consumer Using Semaphore

```
#include <stdio.h>
```

```
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define NUM_PRODUCERS 2
#define NUM_CONSUMERS 2

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void* arg) {
   int id = *(int*)arg;
   int item = 0;

   while (1) {
      item++;

      sem_wait(&empty);
      pthread_mutex_lock(&mutex);

      buffer[in] = item;
      printf("Producer %d produced %d at buffer[%d]\n", id, item, in);
      in = (in + 1) % BUFFER_SIZE;
```

```
        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }
}

void* consumer(void* arg) {
    int id = *(int*)arg;
    int item;

    while (1) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer %d consumed %d from buffer[%d]\n", id, item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(2);
    }
}

int main() {
```

```
pthread_t prod[NUM_PRODUCERS], cons[NUM_CONSUMERS];
int p_id[NUM_PRODUCERS], c_id[NUM_CONSUMERS];

sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0);
pthread_mutex_init(&mutex, NULL);

for (int i = 0; i < NUM_PRODUCERS; i++) {
   p_id[i] = i + 1;
   pthread_create(&prod[i], NULL, producer, &p_id[i]);
}

for (int i = 0; i < NUM_CONSUMERS; i++) {
   c_id[i] = i + 1;
   pthread_create(&cons[i], NULL, consumer, &c_id[i]);
}

for (int i = 0; i < NUM_PRODUCERS; i++)
   pthread_join(prod[i], NULL);

for (int i = 0; i < NUM_CONSUMERS; i++)
   pthread_join(cons[i], NULL);

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

return 0;
```

}

OUTPUT

```
Producer 1 produced 1 at buffer[0]
Producer 2 produced 1 at buffer[1]

Consumer 1 consumed 1 from buffer[0]
Consumer 2 consumed 1 from buffer[1]

Producer 1 produced 2 at buffer[2]
Producer 2 produced 2 at buffer[3]

Consumer 1 consumed 2 from buffer[2]
Consumer 2 consumed 2 from buffer[3]

Producer 1 produced 3 at buffer[4]
Producer 2 produced 3 at buffer[0]

Producer 1 produced 4 at buffer[1]
Producer 2 produced 4 at buffer[2]

Consumer 1 consumed 3 from buffer[4]
Consumer 2 consumed 3 from buffer[0]

Producer 1 produced 5 at buffer[3]
Producer 2 produced 5 at buffer[4]

Consumer 1 consumed 4 from buffer[1]
Consumer 2 consumed 4 from buffer[2]
```

Dining Philosopher

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>


#define N 5 // Number of philosophers


pthread_mutex_t forks[N]; // One mutex per fork

pthread_t philosophers[N];
```

```
void* philosopher(void* num)
{
   int id = *(int*)num;
   int left = id;
   int right = (id + 1) % N;

   while (1)
   {
      printf("Philosopher %d is thinking.\n", id);
      sleep(1);

      if (id % 2 == 0)
      {
         pthread_mutex_lock(&forks[left]);
         printf("Philosopher %d picked up left fork %d.\n", id, left);

         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d.\n", id, right);
      }
      else
      {

         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d.\n", id, right);

         pthread_mutex_lock(&forks[left]);  // FIXED LINE
         printf("Philosopher %d picked up left fork %d.\n", id, left);
      }
```

```
        printf("Philosopher %d is eating.\n", id);
        sleep(2);

        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }

    return NULL;
}

int main()
{
    int i;
    int ids[N];

    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    for (i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }
```

```
    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }

    for (i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

OUTPUT

```
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.

Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.

Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.

Philosopher 0 put down forks 0 and 1.

Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.

Philosopher 2 put down forks 2 and 3.

Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.

Philosopher 4 picked up left fork 4.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
```

Bankers algorithm

```
#include <stdio.h>


int main() {
   int n, m, i, j, k;
   printf("Name:Ojas Manojkumar Saundatti USN:1WA24CS196\n");
   printf("---Bankers algorithm--- \n");
   printf("Enter number of processes: ");
   scanf("%d", &n);


   printf("Enter number of resources: ");
```

```
scanf("%d", &m);

int alloc[n][m], max[n][m], need[n][m];
int avail[m];

printf("Enter Allocation Matrix:\n");
for(i = 0; i < n; i++) {
   for(j = 0; j < m; j++) {
      scanf("%d", &alloc[i][j]);
   }
}

printf("Enter Maximum Demand Matrix:\n");
for(i = 0; i < n; i++) {
   for(j = 0; j < m; j++) {
      scanf("%d", &max[i][j]);
   }
}

printf("Enter Available Resources:\n");
for(i = 0; i < m; i++) {
   scanf("%d", &avail[i]);
}

for(i = 0; i < n; i++) {
   for(j = 0; j < m; j++) {
      need[i][j] = max[i][j] - alloc[i][j];
   }
```

```
}

int finish[n], safeSeq[n], work[m];

for(i = 0; i < n; i++)
   finish[i] = 0;

for(i = 0; i < m; i++)
   work[i] = avail[i];

int count = 0;

while(count < n) {

   int found = 0;

   for(i = 0; i < n; i++) {

      if(finish[i] == 0) {

         int possible = 1;

         for(j = 0; j < m; j++) {

            if(need[i][j] > work[j]) {
               possible = 0;
               break;
            }
```

```
        }

        if(possible) {

            for(k = 0; k < m; k++)
                work[k] += alloc[i][k];

            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
        }
      }
   }

   if(found == 0)
      break;
}

if(count == n) {

   printf("\nSystem is in a safe state.\n");
   printf("Safe sequence is: ");

   for(i = 0; i < n; i++) {

      printf("P%d", safeSeq[i]);

      if(i != n - 1)
```

```
            printf(" -> ");
        }

        printf("\n");
    }
    else {
        printf("\nSystem is NOT in a safe state.\n");
    }

    return 0;
}
```

OUTPUT

```
Name:Ojas Manojkumar Saundatti USN:1WA24CS196
---Bankers algorithm---

Enter number of processes: 5
Enter number of resources: 3

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
```

Deadlock Detection

```
#include <stdio.h>

int main() {

    int n, m, i, j, k;
    printf("\n---Deadlock Detection algorithm--- \n");
    printf("Enter the number of processes: ");
    scanf("%d", &n);
```

```
printf("Enter the number of resources: ");
scanf("%d", &m);

int allocation[n][m];
int request[n][m];
int available[m];

printf("\nEnter the allocation matrix:\n");

for(i = 0; i < n; i++) {

   printf("Process %d: ", i);

   for(j = 0; j < m; j++) {
      scanf("%d", &allocation[i][j]);
   }
}

printf("\nEnter the request matrix:\n");

for(i = 0; i < n; i++) {

   printf("Process %d: ", i);

   for(j = 0; j < m; j++) {
      scanf("%d", &request[i][j]);
   }
}
```

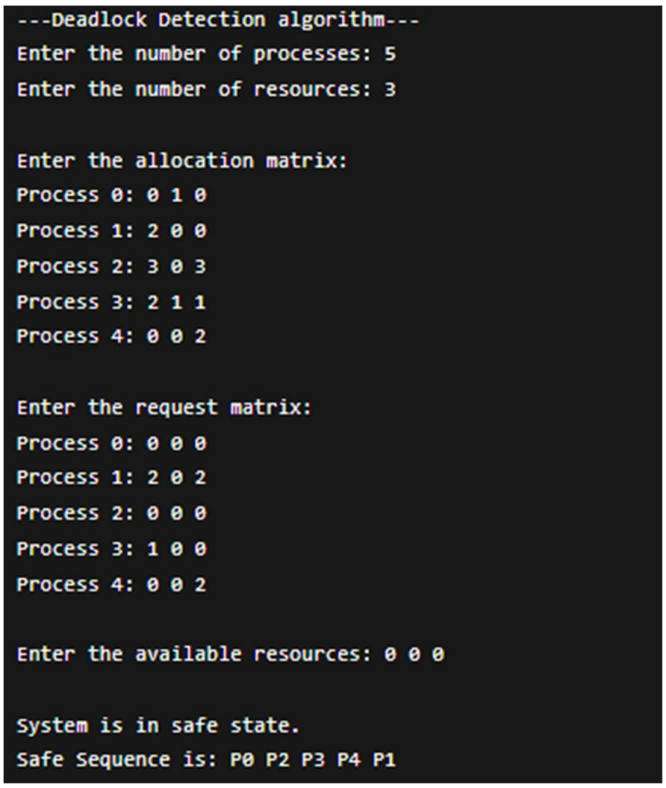
```
printf("\nEnter the available resources: ");

for(i = 0; i < m; i++) {
   scanf("%d", &available[i]);
}

int work[m];
int finish[n];
int safeSeq[n];

for(i = 0; i < m; i++)
   work[i] = available[i];

for(i = 0; i < n; i++) {

   int zero = 1;

   for(j = 0; j < m; j++) {

      if(allocation[i][j] != 0) {
         zero = 0;
         break;
      }
   }

   if(zero)
      finish[i] = 1;
```

```
    else
      finish[i] = 0;
}

int count = 0;

while(count < n) {

   int found = 0;

   for(i = 0; i < n; i++) {

      if(finish[i] == 0) {

         int possible = 1;

         for(j = 0; j < m; j++) {

            if(request[i][j] > work[j]) {
               possible = 0;
               break;
            }
         }

         if(possible) {

            for(k = 0; k < m; k++)
               work[k] += allocation[i][k];
```

```
            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
          }
        }
    }

    if(found == 0)
        break;
}

int deadlock = 0;

for(i = 0; i < n; i++) {

    if(finish[i] == 0) {
        deadlock = 1;
        break;
    }
}

if(deadlock == 0) {

    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < count; i++) {
```

```
            printf("P%d ", safeSeq[i]);
        }

        printf("\n");
    }
    else {

        printf("\nSystem is in deadlock state.\n");
        printf("Deadlocked processes are: ");

        for(i = 0; i < n; i++) {

            if(finish[i] == 0)
                printf("P%d ", i);
        }

        printf("\n");
    }

    return 0;
}
```

OUTPUT



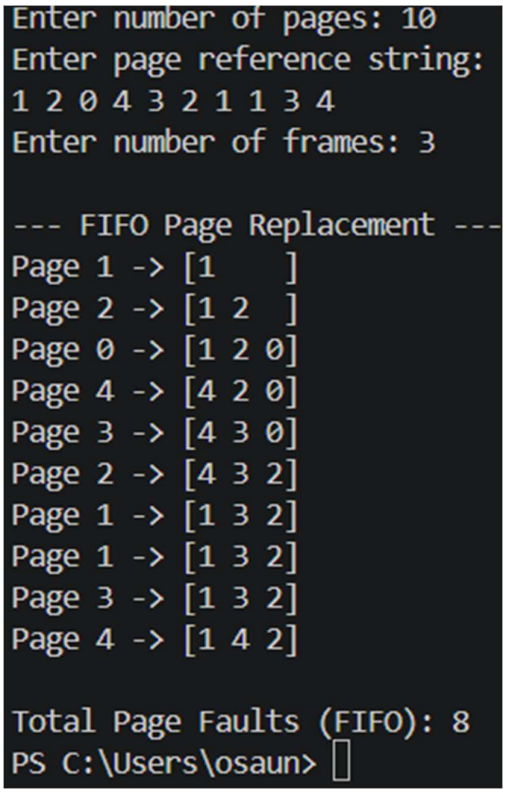
FIFO

```
#include <stdio.h>

int main() {
   int pages[50], frames[10];
   int n, f, i, j;
   int pageFaults = 0;
   int next = 0;
   int found;

   printf("Enter number of pages: ");
   scanf("%d", &n);
```

```
printf("Enter page reference string:\n");
for(i = 0; i < n; i++) {
   scanf("%d", &pages[i]);
}

printf("Enter number of frames: ");
scanf("%d", &f);

for(i = 0; i < f; i++) {
   frames[i] = -1;
}

printf("\n--- FIFO Page Replacement ---\n");

for(i = 0; i < n; i++) {

   found = 0;

   for(j = 0; j < f; j++) {
      if(frames[j] == pages[i]) {
         found = 1;
         break;
      }
   }

   if(found == 0) {
      frames[next] = pages[i];
      next = (next + 1) % f;
```

```
            pageFaults++;
        }

        printf("Page %d -> [", pages[i]);

        for(j = 0; j < f; j++) {
            if(frames[j] != -1)
                printf("%d", frames[j]);
            else
                printf(" ");

            if(j != f - 1)
                printf(" ");
        }

        printf("]\n");
    }

    printf("\nTotal Page Faults (FIFO): %d\n", pageFaults);

    return 0;
}
```

OUTPUT



LRU

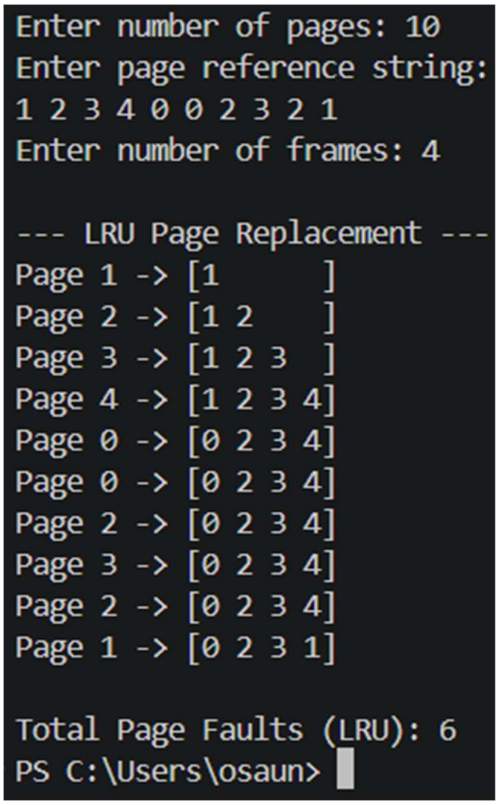
```
#include <stdio.h>


int main() {
   int pages[50], frames[10], time[10];
   int n, f, i, j;
   int pageFaults = 0;
   int counter = 0;
   int found, pos, min;


   printf("Enter number of pages: ");
   scanf("%d", &n);


   printf("Enter page reference string:\n");
   for(i = 0; i < n; i++) {
```

```
    scanf("%d", &pages[i]);
}

printf("Enter number of frames: ");
scanf("%d", &f);

for(i = 0; i < f; i++) {
    frames[i] = -1;
    time[i] = 0;
}

printf("\n--- LRU Page Replacement ---\n");

for(i = 0; i < n; i++) {

    found = 0;

    for(j = 0; j < f; j++) {
        if(frames[j] == pages[i]) {
            found = 1;
            counter++;
            time[j] = counter;
            break;
        }
    }

    if(found == 0) {
```

```
pos = -1;

for(j = 0; j < f; j++) {
   if(frames[j] == -1) {
      pos = j;
      break;
   }
}

if(pos != -1) {
   frames[pos] = pages[i];
   counter++;
   time[pos] = counter;
}
else {
   min = 0;

   for(j = 1; j < f; j++) {
      if(time[j] < time[min]) {
         min = j;
      }
   }

   frames[min] = pages[i];
   counter++;
   time[min] = counter;
}
```

```
            pageFaults++;
        }

        printf("Page %d -> [", pages[i]);

        for(j = 0; j < f; j++) {
            if(frames[j] != -1)
                printf("%d", frames[j]);
            else
                printf(" ");

            if(j != f - 1)
                printf(" ");
        }

        printf("]\n");
    }

    printf("\nTotal Page Faults (LRU): %d\n", pageFaults);

    return 0;
}
```

OUTPUT



ORU

```
#include <stdio.h>


int main() {
   int pages[50], frames[10];
   int n, f, i, j, k;

   int pageFaults = 0;
   int found, pos, farthest, index;

   printf("Enter number of pages: ");
   scanf("%d", &n);

   printf("Enter page reference string:\n");
   for(i = 0; i < n; i++) {
```

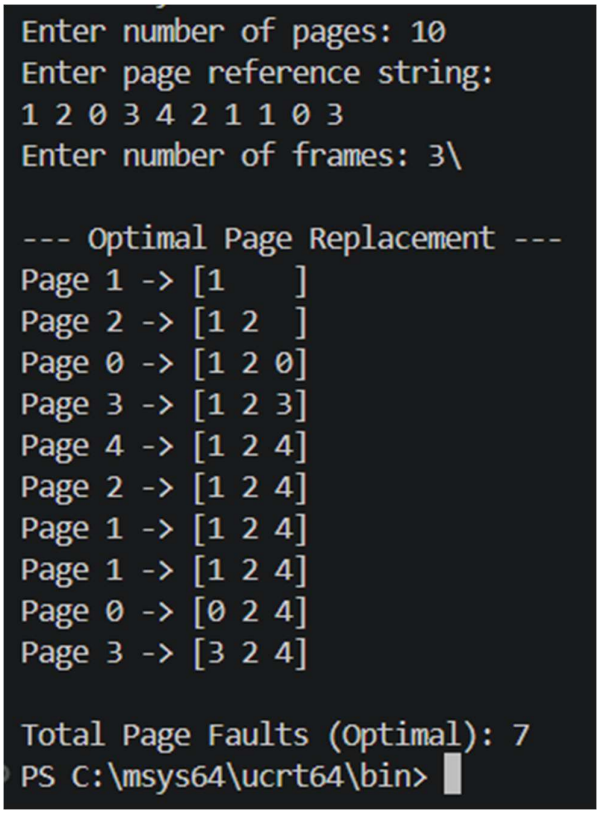
```
    scanf("%d", &pages[i]);
}

printf("Enter number of frames: ");
scanf("%d", &f);

for(i = 0; i < f; i++) {
    frames[i] = -1;
}

printf("\n--- Optimal Page Replacement ---\n");

for(i = 0; i < n; i++) {

    found = 0;

    for(j = 0; j < f; j++) {
        if(frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }

    if(found == 0) {

        pos = -1;

        for(j = 0; j < f; j++) {
```

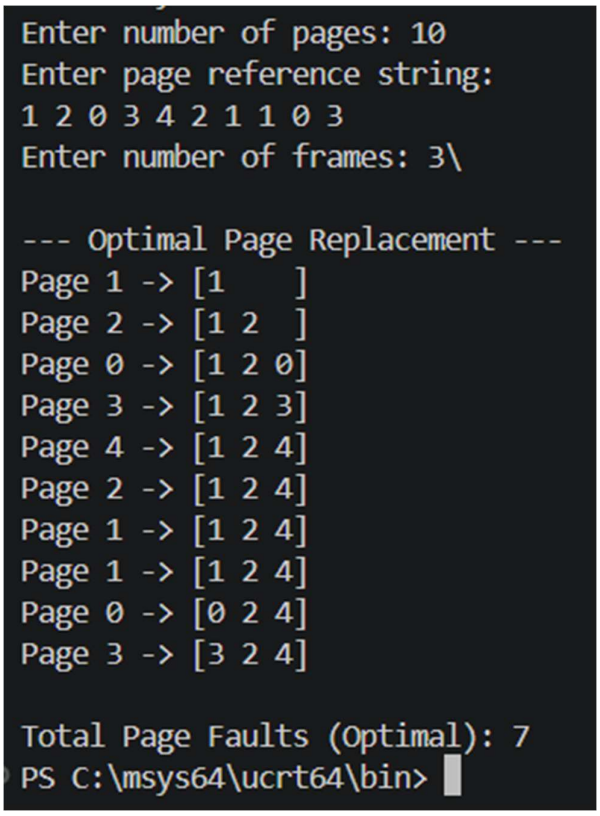
```
    if(frames[j] == -1) {
        pos = j;
        break;
    }
}

if(pos != -1) {
    frames[pos] = pages[i];
}
else {

    farthest = -1;
    index = -1;

    for(j = 0; j < f; j++) {

        int foundFuture = 0;

        for(k = i + 1; k < n; k++) {
            if(frames[j] == pages[k]) {

                if(k > farthest) {
                    farthest = k;
                    index = j;
                }

                foundFuture = 1;
                break;
```

```
            }
          }

          if(foundFuture == 0) {
            index = j;
            break;
          }
        }

        frames[index] = pages[i];
      }

      pageFaults++;
    }

    printf("Page %d -> [", pages[i]);

    for(j = 0; j < f; j++) {

      if(frames[j] != -1)
        printf("%d", frames[j]);
      else
        printf(" ");

      if(j != f - 1)
        printf(" ");
    }
```

```
        printf("]\n");
    }

    printf("\nTotal Page Faults (Optimal): %d\n", pageFaults);

    return 0;
}
```

OUTPUT




Enter number of pages: 10
Enter page reference string:
1 2 0 3 4 2 1 1 0 3
Enter number of frames: 3\
--- Optimal Page Replacement ---
Page 1 -> [1]
Page 2 -> [1 2]
Page 0 -> [1 2 0]
Page 3 -> [1 2 3]
Page 4 -> [1 2 4]
Page 2 -> [1 2 4]
Page 1 -> [1 2 4]
Page 1 -> [1 2 4]
Page 0 -> [0 2 4]
Page 3 -> [3 2 4]
Total Page Faults (Optimal): 7
PS C:\msys64\ucrt64\bin>